

Assignment 1 – Request-reply with at most once semantics

Develop support for request-reply interactions between one or more clients and a server via the following indicative API (as discussed in class).

<i>Client-side API</i>	<code>void init(char *configfname);</code>
	<code>int doRequestReply (int svcid, void *reqbuf, int reqlen, void *rsplib, int *rsplen);</code>
<i>Server-side API</i>	<code>void init();</code>
	<code>int register (int svcid);</code>
	<code>int unregister (int svcid);</code>
	<code>int getRequest (int svcid, void *buf, int *len);</code>
	<code>void sendReply (int reqid, void *buf, int len);</code>

The client-side API logic can run from within the application thread that invokes the `doRequestReply` primitive, blocking the application until the interaction is completed. The server-side API must use an internal thread, which runs concurrently to the application thread(s) invoking the `getRequest` and `sendReply` primitives, to decouple the underlying communication from application-level processing.

The exchange of messages between the client and the server must be performed over UDP/IP. The respective protocol must be independent of the programming language and execution platform of the client and server. You may assume that messages fit into UDP datagrams (no need for fragmentation and reassembly). However, you may not assume an upper bound for the time needed to process a request.

The interaction semantics must be at-most-once even in case of a server reboot. If, after a certain number of attempts, the client-side API does not receive proof of liveness from the server, it must abort the interaction and return an error to the application. Similarly, the server must detect and handle client failures; you are free to pick the client failure model and fault-handling approach.

To circumvent the discovery problem, the contact information (IP address and port number) of the server-side is provided to the client-side in a configuration file. The server can be started on a predefined host and port number, or it can print its contact information to the terminal, so that this can be added manually to the configuration file before starting the client.

Try out your implementation using a simple primality test application. Each client request contains an array of integer values to be tested (the length of the array can vary in different requests). The server checks whether each of these values is a prime number and replies with the result (an array of 0/1 values). The application code must be developed separately from the components that implement the above APIs (which should be treated like libraries). Also, the server application should be multithreaded, with N worker threads.

Conduct experiments to: (a) Measure the delay of a “null” request-reply interaction. (b) Measure the performance of the server as a function of the client request load and the number of worker threads N .

Support for multi-server setup

Extend your implementation to exploit multiple servers offering the same service, while load-balancing between them. This should be introduced as a clean extension of the above protocol and implementation. Also, the APIs should (ideally) remain the same so that the primality test application you have developed above can compile/build with minimal modifications. The contact information of the servers available can be provided to the client-side using the configuration file (one line per server instance).

Conduct experiments to: (a) Measure the additional overhead (if any) for a “null” request-reply interaction. (b) Confirm the desired distribution of client requests to the available servers.

Implementation requirements: The server side must be implemented in C, while the client side must be implemented in a different language (you are free to choose). Data should be properly (de)serialized, without making any assumptions about the internal data representations of the client and server machines. Your implementation should run smoothly in the computer labs with different machines acting as servers/clients.

Submission deadline (see instructions in eclass): Saturday 7 March 2026, 22:00
